TALLER DE PROGRAMACIÓN / CLASE #05

Guía Teórica - Docentes

1. ¿Qué es la repetición condicional?
2. Sintaxis del while
3. Contadores y centinelas: cómo se controla la repetición
4. Break y continue en while
5. El riesgo del bucle infinito
6. While con else y comparación con for
7. Ejemplos teórico-prácticos integradores
99. Ejercitación propuesta

1. ¿Qué es la repetición condicional?

La repetición condicional es una estructura que repite un bloque de instrucciones mientras se cumpla una condición, sin saber de antemano cuántas veces se va a ejecutar. En Python, esta estructura se implementa con while.

Diferencia clave con for (repetición simple):

- for recorre una secuencia ya conocida (una lista, un rango, un string): la cantidad de repeticiones está determinada por el tamaño de esa secuencia.
- while repite mientras una condición sea verdadera: la cantidad de repeticiones puede no conocerse de antemano, y depende de que algo cambie durante la ejecución del bucle.

Se usa while cuando la repetición depende de un evento que ocurre en tiempo de ejecución: que el usuario ingrese cierto valor, que un cálculo llegue a un resultado esperado, que se cumpla una condición externa, etc.

2. Sintaxis del while

Sintaxis general:

```
while condicion:
    # se repite mientras 'condicion' sea True
```

```
contador = 1

while contador <= 5:
    print(contador)
    contador += 1

# Salida: 1 2 3 4 5
```

Idea clave: antes de cada vuelta, Python evalúa la condición. Si es True, ejecuta el cuerpo del bucle; si es False, el bucle termina y el programa continúa con la siguiente instrucción después del while.

3. Contadores y centinelas: cómo se controla la repetición

Para que un while termine correctamente, algo dentro de su condición tiene que cambiar en cada vuelta. Los dos patrones más comunes para lograrlo son:

- Contador: una variable numérica que se incrementa (o decrementa) en cada vuelta hasta alcanzar un límite.
- Centinela: un valor especial que, cuando aparece (por ejemplo, ingresado por el usuario), indica que el bucle debe detenerse.

Patrón contador:

```
numero = 0
while numero < 3:
    print("Vuelta número", numero)
    numero += 1
```

Patrón centinela (flags):

```
respuesta = ""
while respuesta != "salir":
    respuesta = input("Escribí un comando (o 'salir' para terminar): ")
    print("Ingresaste:", respuesta)
```

Nota: Idea clave para remarcar en clase: si la variable de la condición nunca cambia dentro del cuerpo del bucle, el while nunca termina.

4. break y continue en while

Funcionan igual que en for, pero cobran especial importancia en while porque muchas veces se usan para expresar la condición de corte de forma más clara que en el propio while.

break: corta el bucle inmediatamente.

```
numero = 0

while True:             # condición siempre verdadera: el corte lo maneja
el break
    if numero == 5:
        break
    print(numero)
    numero += 1
```

continue: salta a la siguiente vuelta sin ejecutar el resto del cuerpo.

```
numero = 0

while numero < 10:
    numero += 1
    if numero % 2 == 0:
```

```
        continue
    print(numero)   # solo imprime los impares
```

Nota: while True junto con un break interno es un patrón muy usado cuando la condición de corte es más natural de expresar dentro del cuerpo del bucle que en el encabezado (por ejemplo, validar una entrada del usuario).

5. El riesgo del bucle infinito

Un bucle infinito ocurre cuando la condición del while nunca se vuelve False, por lo que el programa queda repitiendo el bloque para siempre (o hasta que se lo interrumpe manualmente).

```
# Ejemplo de bucle infinito (¡NO ejecutar sin cuidado!)
contador = 0
while contador < 5:
    print("Esto nunca termina...")
    # falta "contador += 1" -> la condición nunca cambia
```

Buenas prácticas:

- Antes de ejecutar un while, verificar que exista al menos una instrucción dentro del cuerpo que modifique la variable de la condición.
- Si se usa while True, asegurarse de que exista un break alcanzable.
- En caso de quedar atrapado en un bucle infinito durante las prácticas, se puede interrumpir la ejecución con Ctrl + C (o el botón de detener del entorno que se esté usando).
- Utilizar división en subtareas de representación / abstracción.

6. while con else y comparación con for

Al igual que for, while admite una cláusula else opcional: se ejecuta una sola vez, cuando la condición del while se vuelve False de forma natural (es decir, si el bucle no fue interrumpido con break).

```
intentos = 0
clave_correcta = "1234"
clave_ingresada = ""

while intentos < 3:
    clave_ingresada = input("Ingrese la clave: ")
    if clave_ingresada == clave_correcta:
        print("Acceso concedido")
        break
    intentos += 1
else:
    print("Se agotaron los intentos. Acceso denegado")
```

Cuadro comparativo para cerrar el tema:

Aspecto	for	while
Se usa cuando...	se conoce la secuencia a recorrer	se desconoce cuántas veces se repetirá
Control de corte	tamaño del iterable	una condición booleana
Riesgo principal	ninguno (siempre termina)	bucle infinito si la condición nunca cambia
Típico caso de uso	recorrer listas, strings, rangos	validar entradas, esperar un evento, menús

Ejemplos teórico-prácticos integradores

Ejemplo 1 – while con contador simple

```
n = 1
while n <= 5:
    print(n, "al cuadrado es", n ** 2)
    n += 1
```

Ejemplo 2 – while con centinela (validación de entrada)

```
numero = -1
while numero < 0:
    numero = int(input("Ingrese un número positivo: "))

print("Gracias, ingresaste:", numero)
```

Ejemplo 3 – while True + break (patrón de menú)

```
while True:
    opcion = input("Escriba 'salir' para terminar: ")
    if opcion == "salir":
        print("Programa finalizado")
        break
    print("Comando no reconocido, intente de nuevo")
```

Ejemplo 4 – Acumulación con while (equivalente al patrón visto en for)

```
suma = 0
numero = 1

while numero <= 10:
    suma += numero
    numero += 1

print("La suma de 1 a 10 es:", suma)
```

Ejemplo 5 — while con else (intentos limitados)

```
intentos = 0
adivinado = False
numero_secreto = 7

while intentos < 3 and not adivinado:
    intento = int(input("Adiviná el número (1-10): "))
    if intento == numero_secreto:
        print("¡Correcto!")
        adivinado = True
    intentos += 1
else:
    if not adivinado:
        print("Se acabaron los intentos. El número era", numero_secreto)
```

99. Ejercitación propuesta

EJ01) Usando while, imprimí los números del 1 al 10.

EJ02) Escribí un while que pida un número por teclado hasta que el usuario ingrese exactamente el número 0 (patrón centinela).

EJ03) Usando while, calculá la suma de los números pares entre 1 y 20.

EJ04) Escribí un while que le pida al usuario una contraseña hasta que la ingrese correctamente (contraseña fija: "python123").

EJ05) Usando while y break, escribí un programa que recorra números desde 1 y se detenga apenas encuentre el primer número divisible por 13 y por 7 a la vez.

EJ06) Usando while y continue, imprimí los números del 1 al 30, salteando los múltiplos de 5.

EJ07) Definir una función contar_digitos(numero) que use while para contar cuántos dígitos tiene un número entero positivo, usando división entera.

EJ08) Escribí un programa con while que simule un cajero automático simple: le pida un monto a retirar y repita la solicitud mientras el monto sea mayor al saldo disponible (saldo = 5000), mostrando el saldo restante al finalizar.

EJ09) Combinando while con else, escribí un programa que le dé al usuario 3 intentos para adivinar un número entre 1 y 5, y muestre un mensaje distinto si acierta o si se queda sin intentos.

EJ10) Definir una función es_primo(numero) que use while para determinar si un número es primo (probando divisores desde 2 hasta la raíz cuadrada del número), y luego usá esa función dentro de otro while que imprima todos los números primos entre 2 y 50.

Ejercicio Integrador clase 5 :

Estás desarrollando el menú interactivo de acceso y compras para el sistema digital de un kiosco escolar. El programa debe verificar la identidad del alumno y luego permitirle armar su pedido.

1. Definir una función iniciar_sesion() que le solicite al usuario su PIN de seguridad (por defecto: "1234"). El usuario tiene un máximo de 3 intentos para ingresarlo correctamente.
2. Dentro de la función, utilizá un bucle while con un contador para llevar el registro de los intentos.
3. Si el usuario ingresa el PIN correcto, mostrá un mensaje de bienvenida y utilizá break para interrumpir el bucle de validación.
4. Si el usuario agota sus 3 intentos sin éxito, utilizá la cláusula else del while para imprimir "Cuenta bloqueada por seguridad" y finalizá la ejecución del programa.
5. Si el inicio de sesión fue exitoso, el sistema debe pasar a la toma de pedidos. Inicializá una lista vacía carrito = [] y una variable total = 0.
6. Implementá un patrón centinela usando un bucle while que le pida al usuario ingresar el nombre de un producto, o la palabra "pagar" para terminar la compra.
7. Si el usuario ingresa "pagar", el bucle de compras debe detenerse inmediatamente.
8. Si el usuario ingresa un producto no permitido por las reglas de la escuela (por ejemplo, "energizante"), el sistema debe mostrar un mensaje de advertencia ("Producto restringido") y utilizar continue para saltar a la siguiente vuelta del bucle sin sumarlo al carrito ni pedir su precio.
9. Si el producto es válido, pedile al usuario que ingrese el precio del mismo, agregá el nombre del producto a la lista carrito y sumá el precio a la variable total.
10. Al finalizar el bucle (cuando se elige "pagar"), mostrá un ticket de compra recorriendo la lista carrito para enumerar los productos elegidos y mostrá el monto total final a abonar.